

Сервис детектирования и обезличивания ПД в PostgreSQL

Оглавление

Назначение
Исходный код и технологии
Производительность
30 мар 2022
25 сен 2023
15 окт 2023
Требования по обезличиванию ПД
Требования к процессу обезличивания
Требования к обезличенным ПД
Детектирование ПД
Типы детектируемых данных
Обезличивание ПД
Правила обезличивания ПД
Техническое описание
Структура папок проекта
Особенности работы сервиса
Общие положения (детектирование и обезличивание)
Обезличивание ПД
Замечания по оптимизации скорости работы
Почему сделано своё решение вместо готового?
Стоимость владения
Планируемые доработки
Ссылки по теме
Ресурсы внешние (публичные)
Ресурсы для внутреннего использования

Назначение

Сервис предназначен для автоматического детектирования и обезличивания персональных данных (ПД) в БД PostgreSQL, согласно требованиям законодательства и компании по защите информации. При обезличивании ПД заменяются на анонимные так, чтобы по ним нельзя было однозначно идентифицировать конкретного человека. Такой процесс ещё называют деперсонализацией.

Пример сценария использования — еженедельное обновление тестовой БД актуальными данными с производственной БД.

Исходный код и технологии

GIT репозиторий с исходным кодом: https://git.rabota.space/rdw/rabota/db_utils/-/tree/master/depers

Разработчик: @Ринат Мухтаров (2021 — 2023, ДЗО Работа.Ру)

Технологии и минимальные версии:

- **Ubuntu 18+** или совместимая с ней ОС (GNU/Linux)
- **PostgreSQL 12+** с [PL/pgSQL](#) и расширениями [pgcrypto](#) и [citext](#) (поставляется вместе с БД "из коробки")

- [psql 12+](#)
- [Bash 4.4+](#)

Отсутствие других языков программирования и [CLI](#) вместо [GUI](#) снижает сложность ПО и порог входа другими разработчиками, увеличивает надёжность.

Производительность

Метрики приведены для БД **v3_rabota_test** (размер около 700G) на 30 мар 2022:

1. Для всех таблиц (817 шт.) и колонок типы данных детектируются за **6 минут**.
2. Далее **317 млн.** значений с ПД обезличиваются (в 24 потока) за **7-8 часов** (11,000–12,500 значений в секунду)

Динамика изменения количества ПД по итогам доработок сервиса и изменений данных в БД

30 мар 2022

data_type	tables_count	columns_count	estimate_rows	estimate_percent
email	37	42	91,208,077	28.74
phone	41	51	156,960,673	49.46
birth_date	5	5	69,184,619	21.8
TOTAL	73	98	317,353,369	100

25 сен 2023

data_type	tables_count	columns_count	estimate_rows	estimate_percent
email	46	57	138,308,104	23.99
phone	75	102	361,706,092	62.75
birth_date	4	4	76,412,213	13.26
TOTAL	101	163	576,426,409	100

15 ОКТ 2023

data_type	tables_count	columns_count	estimate_rows	estimate_percent
-----------	--------------	---------------	---------------	------------------

email	61	81	219,386,608	40.27
phone	79	103	248,972,666	45.7
birth_date	4	4	76,468,389	14.04
TOTAL	113	188	544,827,663	100

Информация о тестовом сервере

```

1  $ lscpu
2  Architecture:          x86_64
3  CPU op-mode(s):        32-bit, 64-bit
4  Byte Order:             Little Endian
5  CPU(s):                 80
6  On-line CPU(s) list:    0-79
7  Thread(s) per core:     1
8  Core(s) per socket:     40
9  Socket(s):              2
10 NUMA node(s):           2
11 Vendor ID:              GenuineIntel
12 CPU family:             6
13 Model:                  79
14 Model name:              Intel(R) Xeon(R) CPU E5-2698 v4 @ 2.20GHz
15 Stepping:               1
16 CPU MHz:                2199.996
17 BogomIPS:               4399.99
18 Hypervisor vendor:      KVM
19 Virtualization type:    full
20 L1d cache:              32K
21 L1i cache:              32K
22 L2 cache:               4096K
23 L3 cache:               16384K
24 NUMA node0 CPU(s):      0-39
25 NUMA node1 CPU(s):      40-79
26 Flags:                   fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
                          mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb
                          rdtscp lm constant_tsc arch_perfmon rep_good nopl xtopology cpuid
                          tsc_known_freq pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic
                          movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor
                          lahf_lm abm 3dnowprefetch cpuid_fault invpcid_single ssbd ibrs ibpb stibp
                          fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invpcid rtm rdseed adx
                          smap xsaveopt arat umip md_clear arch_capabilities

```

Требования по обезличиванию ПД

Требования к процессу обезличивания

1. Должен быть механизм автоматического детектирования ПД. Вручную просмотреть все таблицы и все колонки в БД — очень долго и бесперспективно, т.к. колонки с ПД будут появляться и исчезать. Изменение схемы БД должно автоматически отслеживаться:
 - а. при появлении новых колонок с ПД они д.б. обнаружены автоматически, а данные в них обезличены;
 - б. при появлении или исчезновении колонок с ПД процедура обезличивания не должна завершаться с ошибкой.
2. БД должна быть обезличена полностью на 100%. Частично обезличенная БД не д.б. доступна в тестовом контуре. Частично обезличенной БД может быть в процессе работы сервиса или после завершения работы сервиса из-за ошибки.
3. Процедура обезличивания не должна завершаться с ошибкой из-за невалидных/некорректных данных в БД из-за техдолга (отсутствие/некорректная валидация на уровне БД). Такие данные непонятно как обезличивать, поэтому они должны пропускаться. Но все подобные ошибки необходимо записывать в журнал для дальнейшего исследования (нужно убедиться, что всё работает корректно).
4. Должен быть механизм отладки и эмуляции процедуры обезличивания ПД. Перед обезличиванием ПД ключевая функциональность сервиса должна каждый раз проверяться автотестами по следующим причинам:
 - а. Обезличивание ПД необратимо. На больших объёмах данных быстрый откат сделать невозможно. Ждать повторения процедуры после исправления ошибки очень долго.
 - б. Изменяется большое количество данных по всей БД. Цена ошибки высокая.
5. Процедура детектирования и обезличивания ПД должна учитывать следующие особенности:
 - а. Некоторые ПД могут храниться по частям в нескольких колонках. Например, номер телефона и ФИО могут храниться в 3-х колонках (это техдолг).
 - б. В одной колонке могут одновременно храниться разные типы ПД (это техдолг). Например, email и номер телефона.
 - с. В одном поле могут храниться несколько значений:
 - i. в массивах
 - ii. в текстовых полях через запятую или другой разделитель (это техдолг)
 - iii. в [JSON](#)
 - iv. в [tsvector](#)
6. После обезличивания ПД:
 - а. Не должны сломаться автотесты прикладного ПО, которое взаимодействует с БД.
 - б. Не должна сломаться, нарушиться или исказиться бизнес-логика прикладного ПО, которое взаимодействует с БД. Для этого часть ПД должна сохраняться без изменений:
 - i. в email — домен
 - ii. в номере телефона — код страны и код зоны (моб. оператора или города)
 - iii. в ФИО — пол, новое ФИО должно выглядеть правдоподобно (а не набором бессмысленных букв)
 - iv. новая дата рождения должна отличаться от старой не более 6 месяцев в большую или меньшую сторону, при этом д.б. учтено ограничение по минимальному возрасту (например, 14 лет)

Требования к обезличенным ПД

1. Обезличенные ПД не должны совпадать с исходными

2. Обезличенные ПД д.б. необратимыми. Т.е. не должно быть возможности их восстановления в исходный вид. Ещё это называют статическим маскированием.
3. Обезличивание ПД д.б. детерминированным. Т.е. для одних и тех же данных на входе д.б. одни и те же данные на выходе. Например, один номер телефона, где бы он не встречался в БД, д.б. заменён на другой постоянный номер телефона.
4. При обезличивании ПД в БД д.б. сохранена целостность данных на уровне БД:
 1. ограничения уникальности данных в колонках
 2. ограничения внешних ключей
 3. валидация данных на соответствие определённому формату ввода
 - При обезличивании ПД в БД д.б. сохранено форматирование данных, т.к. это может повлиять на целостность данных в БД из-за техдолга (отсутствие/некорректная валидация на уровне БД), на который уже заложились. Например, на ограничения уникальности.
 - При обезличивании ПД в БД необходимо проигнорировать ненастоящие ПД (email и номера телефонов), которые используются для тестирования.

Детектирование ПД

Детектирование (ещё это называют профилированием) некоторых типов данных является нетривиальной задачей и поэтому не является на 100% надёжной. Например, номер телефона м.б. в разных форматах, его можно спутать с числовым идентификатором или ИНН. А дата рождения может быть похожа на обычную дату. Поэтому для качественной работы сервиса:

- детектируется гораздо больше типов данных, чем обезличивается
- для некоторых типов данных есть привязка к возможным названиям колонок
- есть список игнорирования названий схем, таблиц, колонок (в т.ч. по регулярным выражениям)

Процесс детектирования по шагам:

1. Для каждой таблицы БД анализируются случайные 50,000 записей.
2. Для каждой колонки отбираются первые 100 уникальных значений (пустые значения игнорируются).
3. Если значения являются составными типами, а именно массивами, списками (текстовые поля с разделителем значений через запятую или другой разделитель), json или tsvector, то их элементы разворачиваются в записи (одна колонка, много строк). При этом берутся только первые 1,000 элементов для составного типа. Затем записи дедуплицируются.
4. В текстовых значениях анализируются только первые 10,000 символов.
5. Для каждого значения детектируются типы данных. Каждому типу присваивается вес. Найденные типы с весом больше порогового значения считаются типами этой колонки. Если тип является ПД, то значения в колонке будут обезличены.
6. При детектировании типов данных дополнительно проверяется наличие последовательности, где значения в колонках увеличиваются или уменьшаются. Такие колонки игнорируются.

Типы детектируемых данных

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
1	email	да	адрес электронной почты	mike.parker@mail.com, 89651234567@ya.ru	да	
2	phone	да	номер телефона	+7 (965) 123-45-67 - т.д. 8 (83166) 1-23-45 с 9 до 18	да	
3	person_name	да	ФИО	- Иванов Михаил Владимирович - Петрова Варвара Васильевна	да	
4	url	нет	URL	https://www.rabota.ru/company/rabota_ru/	нет	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
5	filename	нет	имя файла с путём	2021/10/14/rive-kirill-41458930.jpg /sbs-2022-01/0/sbs-2022-01_87.pdf 2018/09/10/1536582779-12.mp3	да	
6	sql	нет	SQL запрос к PostgreSQL	<pre>select count(*) from t where deleted_at is null</pre>	нет	
7	json	возможно	JSON object/array, который хранится в строковом типе, а не в json или jsonb типе	{key:"value", []} [7, 9.3, {}]	нет	ПД могут быть внутри элементов JSON
8	uuid	нет	UUID (глобальный идентификатор)	04b1ce9d-2ed0-4db8-8aad-87e1784f3fed 00793c7d72994f8a97628de4dc3d60b5	да	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
9	random	нет	токен, хеш и прочий идентификатор (набор букв, цифр, тире и подчёркивания без пробелов)	- o8se5vuynrb89um5tbvpeipero - TYFDGQV1Hq2L920hK0tm 24print_service - sbs-2022-03 - FF00FF - INCIDENT-310 - rsya_tgbr eg_60396311_pd	да	
10	inet	нет	IPv4	109.205.252.192	да	
11	macaddr	нет	MAC адрес	08:00:2b:01:02:03 08:00:2b:01:02:03:04:05	да	
12	crypt	нет	соль и хеш пароля в формате crypt	\$2a\$10\$066WW2C2Qah0X0FkK4j00eyFD3ZfL4gvY0/EBhA0cFt2zzxtI rjd1a	да	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
13	css_color	нет	цвет в CSS (hexadecimal syntax)	#f09 #FF0099 #F09F #ff0099ff	да	
14	inn	да	ИНН (Идентификационный номер налогоплательщика) - юридического лица (10 цифр) или - физического лица (индивидуального предпринимателя) (12 цифр)	7714467185 781304289675	да	
15	bik	нет	БИК (Банковский Идентификационный Код) (9 цифр)	044525225	да	
16	kpp	нет	КПП (код причины постановки на учёт) (9 цифр)	502901001	да	
17	client_account	нет	расчётный счёт (20 цифр)	40702810440260102703	да	
18	correspondent_account	нет	корреспондентский счёт (20 цифр)	30101810400000000225	да	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
19	ogrn	да	ОГРН (основной государственный регистрационный номер) (13 цифр)	1055005177325	да	
20	ogrnip	да	ОГРНИП (основной государственный регистрационный номер индивидуального предпринимателя) (15 цифр)	307760324100018	да	
21	snils	да	СНИЛС (страховой номер индивидуального лицевого счёта) (11 цифр)	11697338589	да	
22	kladr	нет	КЛАДР (классификатор адресов Российской Федерации) (13, 17, 19 цифр)	1234567890123 780000000000172700 1234567890123456789	да	
23	numeric	нет	вещественное число с указанной точностью	123.456 -78.90123	да	
24	int_range	нет	диапазон целых чисел	50000 - 99999	да	
25	birth_date	да	дата рождения	10.09.1977	да	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
26	date	нет	дата в формате DD.MM.YYYY или YYYY-MM-DD	28.05.2019	да	
27	datetime	нет	дата в формате YYYY-MM-DD HH:MM:SS[.NNN] или YYYYMMDDHHMMSS[.NNN]	24.06.2015 16:21 20181213130700	да	
28	has_escape_d_unicode	нет	содержит кавычированную сущность \uXXXX или \XXXXXXXX	\u0442	нет	
29	has_html_entity	нет	содержит HTML сущность	" & > Ö " ˜	нет	
30	has_xml	нет	содержит XML или HTML код	• •
 • • • <!--...--> • <![CDATA[...]]>	нет	

№	Тип данных	ПД?	Описание	Пример	Детектирование списков в текстовых полях?	Примечания
31	has_delimiter	нет	содержит разделитель значений	- запятая (,) - точка с запятой (;) - перенос строки (\r\n) - слово "или" - слово "or" - html тег для переноса строки (
, <p>,)	нет	

Обезличивание ПД

Ограничения на 15 окт 2023:

- Пока не обезличиваются: ФИО, ИНН, ссылка на фотографию
- Пока не обезличиваются ПД в следующих типах данных: в списках (текстовые поля с разделителем значений через запятую или другой разделитель), JSON.

Правила обезличивания ПД

Тип критичных данных	Пример оригинальных данных	Пример обезличенных данных	Правило замены
Email	mike@yandex.ru	20b84c167281f460f6ad229694e8df17@yandex.ru	Имя пользователя (часть email до @) заменяется на md5 хеш от имени пользователя в нижнем регистре, дополненного "солью". Т.о. имя пользователя заменяется на 32 символа, где каждый символ находится в алфавите [a-z0-9]. Домен (часть email после @) остаётся без изменений. В итоге исходный email заменяется на несуществующий (с 99,99% вероятностью) и отправить на него письмо нет смысла (не будет доставлено). "Соль" — это случайный набор символов, который используется для необратимости восстановления имени пользователя по хешу. Перед хешированием каждое имя пользователя дополняется одним и тем же значением соли длиной 32 символа, где каждый символ находится в алфавите печатных символов диапазона ASCII [\x20-\x7e]. Соль генерируется перед обезличиванием БД и удаляется после. Для хешей возможны коллизии. Если email имеет ограничение уникальности, то для дубликата произойдёт ошибка. На практике такая ситуация — редкость. Ошибки с попыткой нарушения ограничения уникальности будут записаны в журнал.

Тип критичных данных	Пример оригинальных данных	Пример обезличенных данных	Правило замены
			Невалидные email заменяются на фиктивные, но валидные email в формате {id}@invalid.email, где {id} — это идентификатор записи (int/bigint primary key) или сгенерированный UUID.

Тип критичных данных	Пример оригинальных данных	Пример обезличенных данных	Правило замены
Номер телефона	+7 (965) 123-45-67 т.д. 8 (83166) 1-23-45 с 9 до 18	+7 (965) 765-43-21 т.д. 8 (83176) 5-43-21 с 9 до 18	Код страны и зоны (мобильной сети или города) остаётся без изменений. Локальная часть в номере телефона заменяется по справочнику замен. Справочник замен создаётся по всему возможному диапазону пула локальной части номера телефона. По сути это таблица с двумя колонками: старое число и новое число. При этом выделяется 3 поддиапазона с учётом количества цифр: 5, 6 и 7 цифр. Новое число создаётся как случайное значение в своём поддиапазоне. Если в локальной части номера телефона > 7 цифр, то заменяется только последние 7 цифр. В итоге есть вероятность замены существующего номера телефона на несуществующий (на который позвонить не получится, т.к. он не обслуживается оператором). Это хорошо. Старое и новое значение в справочнике никогда не совпадает. Обезличивание уникальных номеров телефонов делается в 2 этапа, если номер телефона имеет ограничение уникальности. 1. На первом этапе длина заменённого номера телефона увеличивается на 3 цифры: добавляется временный неиспользуемый (резервный) код страны 210, а для невалидных номеров телефонов — 214. 2. На втором этапе этот временный код удаляется. Ошибки с попыткой нарушения ограничения уникальности будут записаны в журнал.

Тип критичных данных	Пример оригинальных данных	Пример обезличенных данных	Правило замены
			В невалидных неуникальных номерах телефонов каждая цифра заменяются на ноль. Справочник замен генерируется перед обезличиванием БД и удаляется после.
Дата рождения	10.09.1977	22.05.1977	Заменяется по справочнику замен. Справочник замен создаётся по всему возможному диапазону: с 1900 года по н.в.. По сути это таблица с двумя колонками: старая дата и новая дата. Новая дата создаётся как случайное значение от старой даты в диапазоне 1 года (6 месяцев до и 6 месяцев после). Старое и новое значение в справочнике никогда не совпадает. Справочник замен генерируется перед обезличиванием БД и удаляется после.
ФИО мужского пола	Иванов Михаил Владимирович	Петров Николай Александрович	Заменяется на значение по справочнику, пол сохраняется. TODO написать про справочник замен.
ФИО женского пола	Иванова Марина Владимировна	Сидорова Анна Николаевна	Заменяется на значение по справочнику, пол сохраняется. TODO написать про справочник замен.

Техническое описание

Структура папок проекта

Папка или файл	Описание
dev	Набор разных файлов для текущей и будущей разработки, в работе сервиса не используются, могут быть удалены в любой момент
func_main	Основные SQL функции для обезличивания конкретных типов ПД Основные SQL функции для игнорирования конкретных значений ПД
func_utils	Вспомогательные универсальные SQL функции для детектирования типов данных, нормализации, форматирования, грамматического разбора на составляющие (парсинга) и др.
func_utils_dependent	Вспомогательные SQL функции, которые используют функции из папки func_utils
log	Журналы выполнения работы сервиса и выполнения команд в БД. В случае ошибки сервиса помогают найти и устранить причину. Файлы-журналы удаляются автоматически в момент запуска сервиса.
proc_main	Основные SQL процедуры для детектирования типов данных и обезличивания ПД в БД
proc_utils	Вспомогательные универсальные SQL процедуры
depers_real.sql	SQL команды для обезличивания всех ПД в настоящем режиме
depers_test.sql	SQL команды для обезличивания части ПД в режиме тестирования (эмуляции). Никакие данные в БД не изменяются. По сути это автотестирование обезличивания перед настоящим обезличиванием.
detect.sql	SQL команды для детектирования типов данных в БД

Папка или файл	Описание
run.sh	Скрипт для запуска сервиса

Особенности работы сервиса

Общие положения (детектирование и обезличивание)

1. Сервис работает с готовой необезличенной БД. Такая БД может быть создана, например, из SQL дампа или методом репликации.
2. Сервис принимает на входе полностью необезличенную БД, которая потом становится полностью обезличенной. Инкрементальное обезличивание не поддерживается.
3. В процессе работы никакие объекты БД не модифицируются. Исключение — расширение максимальной длины текста для колонок типа **varchar** для таблиц, в которых хранится номер телефона с ограничением уникальности (см. раздел "Правила обезличивания ПД").
4. Для максимальной безопасности и производительности:
 - a. SQL команды выполняются под супер пользователем с ролью **postgres**
 - b. проверки ограничения внешних ключей и пользовательские триггеры отключаются
5. В сервисе можно настраивать:
 - a. игнорирование объектов БД (схем, таблиц и колонок) и данных (см. таблицы **config_raw**, **email_ignore**, **phone_ignore**)
 - b. правила детектирования (обнаружения) и обезличивания ПД (см. таблицу **rule**)
6. Сервис детектирования или обезличивания ПД умеет продолжать свою работу с того места, где она остановилась из-за ошибки или если её принудительно остановили.

Обезличивание ПД

1. Для ускорения обезличивания ПД несколько процессов БД работают параллельно. При этом процессы автоматически адаптируются под нагрузку БД.
 2. После запуска сервис сначала удаляет (если было), а затем создаёт в БД служебную схему **depers**, в которую записываются необходимые для работы объекты БД. После завершения обезличивания БД схема не удаляется, но доступ к ней имеют только пользователи с ролью суперпользователя **postgres**. Однако удаляются все секретные данные: соль и словари замен для обезличивания ПД.
 3. Данные в таблицах обновляются небольшими частями только командами **UPDATE**.
 1. Продолжительность работы команды — несколько секунд.
 2. В случае возникновения ошибки (например, с попыткой нарушения ограничений целостности БД) происходит автоматический откат только небольшой части данных. Далее сервис позиционируется на конкретную запись, в которой возникает ошибка, записывает её в журнал и продолжает обезличивание других записей.
 3. Если количество ошибок для таблицы > **1%** от общего количества записей в ней, то сервис прерывает работу с фатальной ошибкой.
- В процессе работы сервиса в папке **log** в файл-журнал для каждого параллельного процесса БД каждые несколько секунд записывается информация о том, сколько времени прошло с момента запуска, сколько примерно времени осталось до завершения, прогресс выполнения в процентах.
 - Для работающего процесса БД прогресс выполнения в процентах отображается ещё в представлении **pg_stat_activity** в колонке **application_name**

- Перед настоящим обезличиванием сначала происходит тестирование (эмуляция) обезличивания ПД на реальных данных для быстрого выявления ошибок. Для каждой таблицы выполняются несколько команд UPDATE. Т.е. длительность тестирования почти не зависит от количества данных в таблицах, но зависит от количества таблиц. При тестировании никакие данные в БД не изменяются: по сути вместо фиксации транзакции с обезличенными данными делается её откат.
- В случае повторного запуска этапа обезличивания (например, после ошибки) ранее обезличенные данные в таблице БД будут проигнорированы. Т.е. БД можно дообезличить, повторно разворачивать необезличенную БД из дампа или реплики не нужно.

Замечания по оптимизации скорости работы

При обновлении строк командой **UPDATE** время уходит на следующие действия:

Действие	Реализовано	Оптимизация
Удаление и добавление строк	не применимо	HOT (Heap-Only Tuple update) с настроенным fillfactor для таблиц. Настройка д.б. сделана для таблиц в производственной БД.
Перестроение индексов	нет (TODO)	Перед обновлениями в таблице > 80% (?) строк необходимо сначала удалить все неуникальные индексы, а после обновления построить индексы заново.
Выполнение триггеров	да	Действие отключается под ролью суперпользователя. Так же отключение триггеров помогает избежать нежелательных побочных эффектов!
Проверка целостности по внешним ключам (FK)	да	Действие отключается под ролью суперпользователя.

Почему сделано своё решение вместо готового?

Платное готовое ПО использовать не планировали. Лицензия стоит дорого. Оплата по подписке (не единовременная) или зависит от количества обезличенных ПД.

Так же готовое ПО скорее всего не умеет обрабатывать техдолг: разные типы данных в одной колонке (например phone и email), номер телефона в 3-х колонках.

При сборе требований были рассмотрены описания и GUI интерфейсы нескольких платных ПО из документации в открытом доступе.

Изучена документация и исходный код бесплатного ПО, на 30 мар 2022:

1. Из OpenSource есть только [PostgreSQL Anonymizer](#)
2. [Арбитратор](#) из ДЗО "ДомКлик"

Однако, функциональные возможности этого ПО не соответствуют нашим требованиям по обезличиванию ПД.

Стоимость владения

После окончания этапа активной разработки сервиса стоимость его владения минимальная:

1. Не требуется дополнительных серверов или модернизации текущих
2. Отсутствуют дополнительные финансовые расходы (одноразовые или на регулярной основе), связанные с оплатой лицензий платного ПО
3. Требуется небольшая поддержка со стороны разработчика приложений БД:
 1. для исправления возможных ошибок в сервисе
 2. для более тонкой настройки конфигурационных файлов под особенности конкретных схем БД

Планируемые доработки

1. После детектирования ПД отображать список таблиц и колонок с проблемами и рекомендациями по устранению:
 1. Избыточное кодирование в JSON многобайтовых символов в `\uXXXX`, которое занимает лишнее место в БД. Рекомендовать перейти с `json` на `jsonb`.
 2. Разные типы данных (например, `email` и `phone`) в одной колонке. Рекомендовать сделать отдельные колонки.
 3. В одном текстовом поле данные хранятся в виде списка через запятую или другой разделитель. Рекомендовать использовать массив.
 4. Текстовая колонка, в которой хранится JSON. Рекомендовать изменить тип колонки на `json[b]`.
- После автоматического перезапуска процесса, который завершился с ошибкой, ранее обезличенные данные повторно не обезличиваются, но только для типов `phone` и `email`. Пропуск ранее обезличенных данных занимает время. Необходимо попробовать продолжать обезличивание таблицы и колонки со значения `id`, на котором остановился процесс, завершившийся с ошибкой.
- Оптимизация: для колонок с названием `phone_generated_virtual` детектировать ПД только для типа `phone`
- Добавить тип данных `has_email` для обнаружения `email` внутри текста
- При обезличивании ПД показывать % обработанных данных более точно, ориентируясь на примерное кол-во строк в таблицах
- Попробовать в PostgreSQL включить/выключить JIT и замерить длительность выполнения
- Попробовать ускорить выполнение, используя `SET CONSTRAINTS ALL DEFERRED;` (<https://stackoverflow.com/questions/2679854/postgresql-disabling-constraints>)

Ссылки по теме

Ресурсы внешние (публичные)

Альтернативные анонимайзеры (список на  09.09.2025)

1. https://gitlab.com/dalibo/postgresql_anonymizer
2. https://github.com/TantorLabs/pg_anon (документация на русском языке)
3. https://github.com/diffix/pg_diffix/ — PostgreSQL extension for strong dynamic anonymization
4. <https://postgrespro.ru/docs/enterprise/17/pgpro-anonymizer> (для БД PostgreSQL Pro)

Прочее

1. PostgreSQL: How to update large tables
2. <https://stackoverflow.com/questions/8290900/best-way-to-delete-millions-of-rows-by-id/8290958#8290958>
3. <https://dba.stackexchange.com/questions/52517/best-way-to-populate-a-new-column-in-a-large-table/52531#52531>
4. Получение данных о кодах статистики (ОКПО, ОКАТО, ОКФС, ОКТМО, ОКОГУ, ОКОПФ) и перечня форм по ИНН или ОГРН/ОГРНИП
 - а. Что такое ОКПО, ОКАТО, ОКФС, ОКТМО, ОКОГУ, ОКОПФ и их форматы
5. Сбер: Система генерации связанных тестовых данных Synteta
6.  Нормализация и валидация email
7.  Нормализация и валидация номера телефона

Ресурсы для внутреннего использования

1.  Обезличивание ПД в СУБД
2.  Безопасность персональных данных
 - а. Сбер: Обезличивание тестовых стендов v11.pdf
3. Обезличивание данных и синтетическая генерация данных